AutoGrader: A Rule-Driven Evaluator for C Programs

When a class submits programs, someone has to run them, decide whether they are correct, how efficient they are, and whether the code itself is written well. Doing this by hand is slow, inconsistent, and does not scale. The purpose of this assignment is to build an *intelligent agent* that does the whole job on its own: it takes a student's C submission and a set of grading rules the instructor has written, runs the submission safely, checks it against every rule, and returns one overall grade together with feedback that tells the student where the program falls short.

A grade produced by the agent must be **reproducible**: the same submission, judged under the same rules, is expected to receive the same grade every time the agent is run.

1. Objective

Design and implement an agent that grades student C programs automatically, according to rules the instructor supplies. The agent must:

1. **run** the submission safely, under fixed time and memory limits, and survive programs that crash or never halt;
2. **read** the grading rules from a file kept *outside* the agent's source code, so that adding or changing a rule does not require editing the agent;
3. **evaluate** each metric with its own analyzer, and combine the analyzer results into a single overall grade using the weights the rules specify;
4. **produce feedback** that, for an incorrect submission, includes a specific input that exposes the failure and the expected output beside it; The feedback can highlight any possible memory leaks from the code (eg. valgrind/asan). Also you may consider profiling of the code (eg. oprofile/perf)
5. **flag** suspected copying from online resources or LLM's or others.
6. State the **usefulness** of the agent in real-life settings and its **limitations**, with 3-4 concrete examples of each.

Everything about *how* the agent works internally is yours to decide and defend: the method behind each analyzer, how complexity is estimated, how the analyzers are coordinated, and the way feedback is generated. What must not change is the **contract**: same submission + same rules → same grade.

Requirements

Your agent must:

1. Run a C submission safely, under fixed time and memory limits, and survive programs that crash or never halt.
2. Take its instructions from a set of rules the instructor defines which metrics to grade, and the threshold and weight for each kept outside the code, so the rules can change without editing the agent.
3. Evaluate each metric with its own analyzer, coordinated by the agent. The set of metrics is open: correctness, time efficiency against the required bound, code structure, and originality are examples, and the rules may add others.
4. Run end to end unattended: given a submission and the rules, produce one reproducible overall grade and the feedback, with no or minimal manual steps in between.
5. For an incorrect submission, produce a specific input that exposes the failure, with the expected output beside it along with the explanations, generate counter test cases where the code will fail, take all the corner cases into account.
6. Specify the usefulness (real-life implementation) and the limitations of the agent, with 3-4 examples of each. Each limitation must name a concrete case where the agent gives a wrong or unreliable verdict

Deliverables

- A complete, working agent that, given a submission and the rules, runs the full evaluation unattended and outputs the grade and feedback.
- The rules it reads, metrics, thresholds, and weights in a form the instructor can edit without touching code, with at least one example rule set.
- Your agent's output on the sample submissions provided.
- A "Usefulness and Limitations" section in the final report

Tools you may use

The methods are yours, but you need not build everything from scratch. Some tools that fit each task:

Ollama (ollama.com) — not an API service; a local model runner. ollama run llama3.1 starts a local HTTP server on port 11434 that speaks the same JSON shape. Fully offline, fully free, no rate limits, but needs a machine with enough RAM.

Codacy — free for open-source repositories; exposes results and issue lists through a REST API at app.codacy.com/api/v3. Comparable coverage to SonarCloud for C.

Example:

The problem

Read two integers on a single line separated by whitespace and print their sum followed by a newline.

Sample input:

```
7 5
```

Sample output:

```
12
```

The student's submission

The student uploads a single file add.c:

```
#include <stdio.h>
int main(){
  int a,b;
  scanf("%d %d",&a,&b);
  printf("%d\n",a+b);
  return 0;
}
```

It works for common inputs but has visible weaknesses: everything is in main(), variable names are single letters, there are no comments, and INT_MAX + 1 will overflow silently.

The rules the instructor wrote

The instructor drops the following rules file next to the submission. Weights sum to 1.00

```
"problem":        "add_two_integers",
"time_limit_s":     1,
"memory_limit_mb":  64,
"metrics": [
  {
    "name":     "correctness",
    "analyzer": "io_tests",
    "weight":   0.20,
    "params":   { "testset_dir": "tests/add/" }
  },
  {
    "name":     "syntax",
    "analyzer": "compiler_warnings",
    "weight":   0.10,
    "params":   { "flags": "-Wall -Wextra -Werror -std=c11" }
  },
```

```
{
 "name":     "structure",
 "analyzer": "static_analysis",
 "weight":   0.10,
 "params":   { "max_cyclomatic": 8, "max_function_length": 40 }
},
{
 "name":     "modularity",
 "analyzer": "function_split",
 "weight":   0.10,
 "params":   { "expect_helpers": true, "min_functions": 2 }
},
{
 "name":     "logical",
 "analyzer": "edge_case_runner",
 "weight":   0.10,
 "params":   { "cases": ["negatives", "zero", "int_overflow"] }
},
{
 "name":     "complexity",
 "analyzer": "complexity_estimator",
 "weight":   0.10,
 "params":   { "required": "O(1)", "sizes": [1, 10, 100] }
},
{
 "name":     "memory_safety",
 "analyzer": "sanitizer_run",
 "weight":   0.10,
 "params":   { "flags": "-fsanitize=address,undefined" }
},
{
 "name":     "readability",
 "analyzer": "llm_review",
 "weight":   0.10,
 "params":   { "model": "llama-3.3-70b-versatile",
              "temperature": 0, "seed": 42 }
},
{
 "name":     "originality",
 "analyzer": "similarity_flag",
 "weight":   0.10,
 "params":   { "corpus_dir": "submissions/",
              "flag_threshold": 0.85 }
```

```
  }
]
}
```

The main agent's first pass

Before any specialist runs, the main agent (the coordinator) does a cheap sanity sweep so that later analyzers are not run on a submission that cannot even be executed.

Compile check. Run gcc -O2 -Wall -Wextra -std=c11 in the sandbox. The file compiles with one warning; compilation succeeds.

Smoke run. Feed the compiled binary the sample input under a 1-second wall-clock timeout and a 64 MB memory cap. It exits cleanly in a few milliseconds — no infinite loop, no crash.

Static sanity. Confirm a main() is present, at least one scanf/reader is called, at least one printf/writer is called, and the source contains a return statement in main.

Because the first pass passes, the main agent proceeds to dispatch. Had the smoke run *timed out*, the agent would have set a suspected_infinite_loop flag on the report, skipped the execution-based analyzers (correctness, logical, complexity, memory safety), still run the source-only analyzers (syntax, structure, modularity, readability, originality), and emitted a partial report.

Dispatch to expert analyzers

The main agent invokes each expert once, in parallel where possible, forwarding the source file, the compiled binary, and the analyzer's own params slice. Each expert returns a score, its raw evidence, and a short feedback string. The results collected on this submission:

Metric	Weight	Expert analyzer	Score	Verdict
correctness	20 %	io_tests (hidden test runner)	18/20	8 of 10 tests pass
syntax	10 %	compiler_warnings (gcc -Wall -Wextra)	9/10	one -Wunused-parameter warning
structure	10 %	static_analysis (cppcheck + lizard)	6/10	no headers, single-block code
modularity	10 %	function_split	3/10	only main(); no helper
logical	10 %	edge_case_runner	8/10	overflow on INT_MAX+1 not handled

complexity	10 %	complexity_estimator (timing curve)	10/10	O(1) confirmed
memory_safety	10 %	sanitizer_run (ASan + UBSan)	10/10	no diagnostics
readability	10 %	llm_review (Groq, Llama 3.3 70B, T=0)	5/10	cramped, no comments, terse names
originality	10 %	similarity_flag	10/10	max similarity 0.42, not flagged

Weighted total: **18 + 9 + 6 + 3 + 8 + 10 + 10 + 5 + 10 = 89 / 100**.

Collation: the final report

The main agent writes two files. The JSON report is for machines; the text report is what the student reads.

reports/add.json (excerpt):

```
{
"submission":  "add.c",
"rules_hash":  "sha256:5f2c...b9e1",
"grade":       89.0,
"first_pass":  { "compiled": true, "smoke_ok": true,
                 "suspected_infinite_loop": false },
"metrics": [
  { "name": "correctness", "score": 0.90, "weight": 0.20,
    "contribution": 18.0,
    "evidence": {
      "tests_passed": 8, "tests_total": 10,
      "failing":  [
        { "input": "2147483647 1",
          "expected": "-2147483648", "got": "" ,
          "note": "runtime error: signed integer overflow" }
      ]
    },
    "feedback": "8/10 tests pass. Fails on INT_MAX + 1: signed overflow." },
  { "name": "modularity", "score": 0.30, "weight": 0.10,
    "contribution": 3.0,
    "evidence": { "functions_defined": 1, "expected_min": 2 },
    "feedback": "Only main() is defined. Extract an add(int,int) helper." },
  { "name": "readability", "score": 0.50, "weight": 0.10,
    "contribution": 5.0,
```

```
    "evidence": {
      "model": "llama-3.3-70b-versatile", "temperature": 0, "seed": 42,
      "raw_response": "{ \"score\": 5, \"reasons\": [ ... ] }"
    },
    "feedback": "Single-letter names, no comments, missing whitespace." }
  /* other metrics elided for brevity */
 ],
"flags":     [],
"usefulness_limitations": {
 "usefulness": [
   {
     "use_case": "Coding-interview and hiring screens",
     "example":  "A company runs 500 candidate submissions for an 'add two integers safely'
task. The agent ranks them by correctness, edge-case handling and memory safety, and gives an
explainable per-metric score.",
     "benefit":  "Fast first-round filtering where every rejection can be justified metric by metric."
   },
   {
     "use_case": "Plagiarism and LLM-copy triage for instructors",
     "example":  "Two submissions reach a similarity of 0.93 against the 0.85 threshold, so the
agent raises a flag in the report. The instructor reviews only those pairs instead of all 300.",
     "benefit":  "Saves review time by narrowing manual checking to suspicious cases."
   }
 ],
 "limitations": [
   {
     "limitation": "Tests cannot prove correctness",
     "example":    "A submission passes all 10 hidden tests but fails when the input has extra
whitespace or a trailing newline that no test covers. The agent still reports 10/10.",
     "mitigation": "The agent should create its own extra tricky test inputs and compare the
student's answers with the answers of a correct program."
   }
 ]
},
 "tool_versions": {
  "gcc": "13.2.0", "cppcheck": "2.14.1", "lizard": "1.17.10",
 }
}
```


reports/add.txt (what the student sees):

```
===== Grading report for add.c =====
Overall grade: 89.0 / 100
Metric          Weight  Score   Contribution
correctness      20     18/20   18.0    8 of 10 tests pass
```

```
syntax           10      9/10    9.0    -Wunused-parameter x1
structure        10      6/10    6.0    single-block code, no headers
modularity       10      3/10    3.0    only main() defined
logical          10      8/10    8.0    INT_MAX + 1 overflow not handled
complexity       10     10/10   10.0    O(1) confirmed
memory_safety    10     10/10   10.0    ASan/UBSan clean
readability      10      5/10    5.0    cramped, no comments
originality      10     10/10   10.0    max similarity 0.42 (safe)
--- Where you failed ---
Test 07  INPUT:    2147483647 1
        EXPECTED: -2147483648
        GOT:      (runtime error: signed integer overflow)
        WHY:      a + b overflows on 32-bit signed ints. Use long long
                  or check that b <= INT_MAX - a before adding.
--- Suggestions to raise the grade ---
- Extract  int add(int a, int b)  into its own function; call it from main.
- Rename a,b to lhs,rhs (or similar); add a one-line comment above main().
- Handle overflow explicitly with a wider integer type.
--- Usefulness & limitations of this evaluation ---
Useful for:
  1. Coding-interview and hiring screens
     Example: a company runs 500 candidate submissions for an
     "add two integers safely" task. The agent ranks them by
     correctness, edge-case handling and memory safety, and gives
     a score for each metric.
     Benefit: fast first-round filtering where every rejection can
     be explained metric by metric.

  2. Plagiarism and LLM-copy triage for instructors
     Example: two submissions reach a similarity of 0.93, above the
     0.85 threshold, so the agent flags them. The instructor checks
     only those pairs instead of all 300 submissions.
     Benefit: saves review time by limiting manual checking to
     suspicious cases.

Limits of this report:
  1. Tests cannot prove correctness
     Example: a submission passes all 10 hidden tests but fails when
     the input has extra whitespace or a trailing newline that no test
     covers. The agent still reports 10/10.
     Fix: the agent creates its own extra tricky test inputs and
     compares the student's answers with the answers of a correct
     program.
```

Submission plan:

- You can work this assignment in a group with no more than 4 students.
- You will get around 4 weeks time to complete this.
- Every week there will be an evaluation on your progress.

- At the end, you need to submit a report, code base (github link with appropriate installation guide) and a 15-20 minutes video uploaded on youtube. We will let you know how to submit it.
- Before the next lab, you need to upload a report (pdf file namely agentic01.pdf) about the plan for your implementation, distribution of work loads among the members of the group, progress so far and future plan with timeline. Only one member of the group will submit this file.
- You can use any LLMs. However, you must own the code and report. There will be severe penalties if it is found that the output from the generative AI tools has been used blindly.